

```

1. ИСХОДНЫЙ КОД (Императивная «каша», логика и DOM перемешаны)
// =====
import '@splidejs/splide/dist/css/splide-core.min.css';
import '../scss/style.scss';
import Splide from '@splidejs/splide';

// КОСЯК: Весь MVC перемешан в одном событии. Логика валидации намертво завязана на DOM-элементы.
document.getElementById('appointment-form').addEventListener('submit', (event) => {
  event.preventDefault();
  const fields = document.querySelectorAll('input');

  // КОСЯК: Ручной сброс текста в DOM-узлах. При добавлении нового поля придется дописывать строки руками.
  document.getElementById('name-error').innerText = '';
  document.getElementById('phone-error').innerText = '';
  document.getElementById('email-error').innerText = '';
  document.getElementById('success-message').innerText = '';

  let isValid = true;

  // КОСЯК: Слепая проверка всех инпутов без разбора их назначения.
  fields.forEach((field) => {
    if (!field.value.trim()) {
      isValid = false;
      field.classList.add('is-invalid');
    } else {
      field.classList.remove('is-invalid');
    }
  });

  // КОСЯК: Бизнес-логика (регулярки) и UI (innerText) выполняются одновременно. Нет валидации "на лету" при вводе.
  const name = document.getElementById('name').value;
  if (name.trim() === '') {
    document.getElementById('name-error').innerText = 'field cannot be empty';
    isValid = false;
  }

  const phone = document.getElementById('phone').value;
  const phonePattern = /^+\d{10}$/;
  if (!phonePattern.test(phone)) {
    document.getElementById('phone-error').innerText = 'the correct phone format +79999999999';
    isValid = false;
  }

  const email = document.getElementById('email').value;
  const emailPattern = /^+\S+@\S+\.\S+$/;
  if (!emailPattern.test(email)) {
    document.getElementById('email-error').innerText = 'incorrect email format';
    isValid = false;
  }

  if (isValid) {
    // КОСЯК (Race Condition): Кнопка отправки не блокируется. Пользователь может кликнуть 10 раз и спамить сервер.
    document.getElementById('success-message').innerText = 'The form has been successfully submitted!';
    document.getElementById('appointment-form').reset();
    setTimeout(() => {
      document.getElementById('success-message').innerText = '';
    }, 1000);
    // + fetch или XMLHttpRequest
  }
});

// ИНФРАСТРУКТУРА: Анимации и слайдеры (Оставляем как есть, MVC тут не нужен)
const observer = new IntersectionObserver((entries) => {
  entries.forEach((entry) => {
    entry.target.classList.toggle('hide', !entry.isIntersecting);
    if (entry.isIntersecting) observer.unobserve(entry.target);
  });
}, { threshold: 0.7 });

const configSplideSpeakers = {
  type: 'slide', perPage: 3, autoplay: false, pagination: false, arrows: false, gap: '20px',
  breakpoints: { 1200: { perPage: 2, arrows: true }, 640: { perPage: 1, gap: '0px' } }
};

function initSplide() {
  new Splide('#slider-speakers', configSplideSpeakers).mount();
}

```

```

}

function initAnimation() {
  document.querySelectorAll('.js-animate').forEach((target) => {
    target.classList.remove('js-animate');
    target.classList.add('hide');
    setTimeout(() => { observer.observe(target); }, 500);
  });
}

function handleDOMContentLoaded() {
  const burger = document.getElementById('burger');
  const headerButton = document.getElementById('header-button');
  const nav = document.getElementById('desktop-nav');
  const headerLink = nav.querySelectorAll('a');
  const { body } = document;
  const scrollBtn = document.getElementById('scroll-top-btn');

  burger.addEventListener('click', () => {
    headerButton.classList.toggle('active');
    nav.classList.toggle('active');
    burger.classList.toggle('active');
    body.style.overflow = nav.classList.contains('active') ? 'hidden' : 'auto';
  });

  window.addEventListener('scroll', () => {
    scrollBtn.style.display = window.scrollY > 300 ? 'flex' : 'none';
  });

  scrollBtn.addEventListener('click', () => {
    window.scrollTo({ top: 0, behavior: 'smooth' });
  });

  initSplide();
  initAnimation();
}

window.addEventListener('DOMContentLoaded', () => handleDOMContentLoaded(), { once: true });

```